PATRONES DE DISEÑO EN AIRBNB

Implementación Completa y Análisis Técnico

Estudiantes: Sofia Zambrano Paez · Kennet Mark Olarte

2025

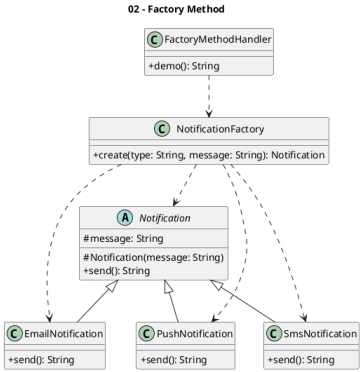
FACTORY METHOD

Patrón de Diseño GoF

Problema en Airbnb

Gestión de múltiples canales de notificación (Email, Push, SMS) sin acoplar el código cliente a las implementaciones concretas.

- **Flexibilidad:** Añadir nuevos canales requiere mínima modificación.
- **Abstracción:** El cliente trata a todas como `Notification`.



Jerarquía de Notificaciones en Airbnb

```java
// FactoryMethodHandler.java
public class FactoryMethodHandler {
    public String demo() {
        NotificationFactory factory = new NotificationFactory();
        String email = factory.create("email", "Reserva confirmada").send();
        String push = factory.create("push", "Nuevo mensaje del anfitrión").send();
        String sms = factory.create("sms", "Pago recibido").send();
        return email + " | " + push + " | " + sms;
    }
}
```

- **Responsabilidad:** Actúa como el punto de entrada desde la API. Coordina la creación mediante la fábrica y recolecta los resultados para el cliente web.
- **Desacoplamiento:** El Handler no conoce las clases `EmailNotification`, etc. Solo interactúa con la fábrica y la abstracción `Notification`.

```java
// Notification.java (Abstract)
public abstract class Notification {
    protected final String message;
    protected Notification(String message) { this.message = message; }
    public abstract String send();
}
// EmailNotification.java (Concrete)
public class EmailNotification extends Notification {
    public EmailNotification(String message) { super(message); }
    public String send() { return "Email enviado: " + message; }
}
```

```java
// NotificationFactory.java
public class NotificationFactory {
    public Notification create(String type, String message) {
        String normalized = type == null ? "" : type.trim().toLowerCase();
        if ("email".equals(normalized)) return new EmailNotification(message);
        if ("push".equals(normalized))  return new PushNotification(message);
        if ("sms".equals(normalized))   return new SmsNotification(message);
        throw new IllegalArgumentException("Tipo no soportado: " + type);
    }
}
```

Entrada (Request JSON)

```
{
  "pattern": "02",
  "input": "Reserva confirmada"
}
```

Resultado en Airbnb

`"Email enviado: Reserva confirmada | Push enviado: ... | SMS enviado: ..."` **Explicación:** El Handler invoca el método `demo()`. La fábrica crea dinámicamente cada objeto. Cada implementación de `send()` añade su propio contexto (prefijo), demostrando el polimorfismo.

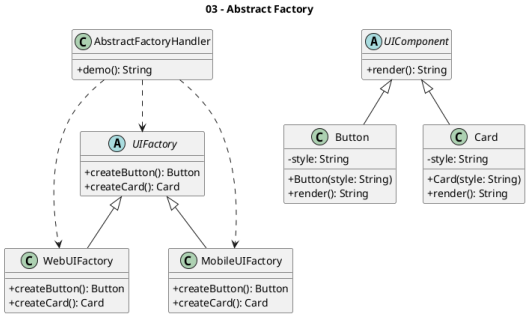
ABSTRACT FACTORY

Patrón de Diseño GoF

Problema en Airbnb

Garantizar la consistencia visual entre componentes (Botones, Tarjetas) según la plataforma (Web vs Móvil).

- **Coherencia:** Evita mezclar estilos web con móviles.



Familias de UI por Plataforma

```java
// AbstractFactoryHandler.java
public class AbstractFactoryHandler {
    public String demo() {
        UIFactory webFactory = new WebUIFactory();
        UIFactory mobileFactory = new MobileUIFactory();
        return webFactory.createButton().render() + " | " + webFactory.createCard().render() + " | "
                + mobileFactory.createButton().render() + " | " + mobileFactory.createCard().render();
    }
}
```

- **Lógica de Demo:** El Handler demuestra la capacidad de cambiar familias enteras instanciando diferentes fábricas concretas que heredan de `UIFactory`.
- **Responsabilidad:** Centraliza la creación de los componentes que luego el front-end web mostrará al usuario.

```java
// UIComponent.java (Abstract Product)
public abstract class UIComponent {
    private final String style;
    protected UIComponent(String style) { this.style = style; }
    public abstract String render();
}
// Button.java / Card.java
public class Button extends UIComponent {
    public Button(String style) { super(style); }
    public String render() { return "Botón " + getStyle(); }
}
```

```java
// UIFactory.java (Abstract Factory)
public abstract class UIFactory {
    public abstract Button createButton();
    public abstract Card createCard();
}
// WebUIFactory.java / MobileUIFactory.java
public class WebUIFactory extends UIFactory {
    public Button createButton() { return new Button("web"); }
    public Card createCard() { return new Card("web"); }
}
```

Entrada (Request JSON)

```
{
  "pattern": "03",
  "input": "demo"
}
```

Resultado en Airbnb

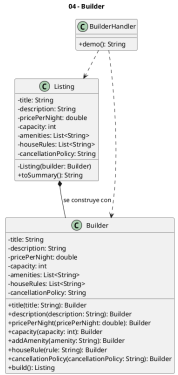
`"Botón web | Tarjeta web | Botón móvil | Tarjeta móvil"` **Explicación:** El Handler alterna entre `WebUIFactory` y `MobileUIFactory`. Cada una garantiza que sus productos (`Button`, `Card`) compartan el mismo estilo ("web" o "móvil") inyectado en el constructor.

BUILDER

Patrón de Diseño GoF

Problema en Airbnb

Creación de anuncios (`Listing`) con parámetros opcionales y listas complejas.



Proceso de Construcción

```java
// BuilderHandler.java
public class BuilderHandler {
    public String demo() {
        Listing listing = new ListingBuilder()
                .title("Apartamento en Barcelona")
                .description("Luminoso y cerca del metro")
                .pricePerNight(120).capacity(4)
                .addAmenity("Wi-Fi").addAmenity("Cocina")
                .build();
        return listing.summary();
    }
}
```

- **Lógica Fluida:** El Handler utiliza la interfaz fluida del Builder para configurar el objeto `Listing` de manera legible y estructurada.
- **Encapsulamiento:** El Handler nunca interactúa directamente con el constructor de `Listing`, protegiendo la integridad de la clase de negocio.

```java
// ListingBuilder.java
public class ListingBuilder {
    private String title;
    private List<String> amenities = new ArrayList<>();
    public ListingBuilder title(String t) { this.title = t; return this; }
    public ListingBuilder addAmenity(String a) { amenities.add(a); return this; }
    public Listing build() { return new Listing(title, ..., amenities, ...); }
}
// Listing.java
public class Listing {
    private final String title;
    public Listing(String t, ..., List<String> a, ...) { this.title = t; ... }
    public String summary() { return title + " | " + ...; }
}
```

Entrada (Request JSON)

```
{
  "pattern": "04",
  "input": "Barcelona Apto"
}
```

Resultado en Airbnb

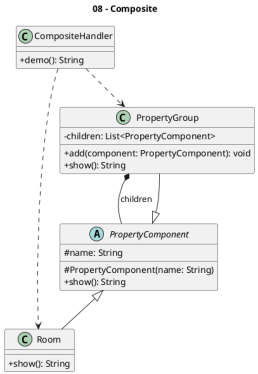
`"Apartamento en Barcelona | ... | 120.0 | 4 | [Wi-Fi, Cocina] | ..."` **Explicación:** El Handler construye el objeto paso a paso. El resultado final muestra un resumen coherente de todos los atributos seteados dinámicamente, confirmando que el Builder transfirió correctamente el estado al objeto `Listing`.

COMPOSITE

Patrón de Diseño GoF

Problema en Airbnb

Representar jerarquías de propiedades (Habitación, Apartamento, Edificio).



Estructura Recursiva

```java
// CompositeHandler.java
public class CompositeHandler {
    public String demo() {
        PropertyGroup apartment = new PropertyGroup("Apartamento Centro");
        apartment.add(new Room("Habitación 1"));
        apartment.add(new Room("Habitación 2"));

        PropertyGroup building = new PropertyGroup("Edificio Principal");
        building.add(apartment);
        building.add(new Room("Habitación Penthouse"));

        return building.show();
    }
}
```

- **Construcción del Árbol:** El Handler crea la estructura jerárquica agregando tanto "Hojas" (`Room`) como "Compuestos" (`PropertyGroup`) al grupo raíz.
- **Uniformidad:** El Handler trata a `building` y `apartment` como `PropertyComponent`.

```java
// PropertyComponent.java
public abstract class PropertyComponent {
    public abstract String show();
}
// Room.java (Leaf)
public class Room extends PropertyComponent {
    public String show() { return name; }
}
// PropertyGroup.java (Composite)
public class PropertyGroup extends PropertyComponent {
    private final List<PropertyComponent> children = new ArrayList<>();
    public void add(PropertyComponent c) { children.add(c); }
    public String show() {
        StringBuilder b = new StringBuilder(name);
        for (PropertyComponent child : children) b.append(" -> ").append(child.show());
        return b.toString();
    }
}
```

Entrada (Request JSON)

```
{
  "pattern": "08",
  "input": "demo"
}
```

Resultado en Airbnb

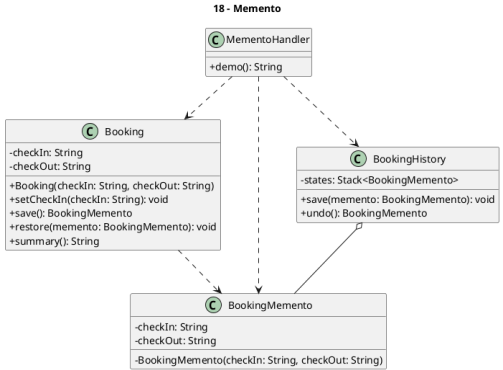
`"Edificio Principal -> Apartamento Centro -> Habitación 1 -> Habitación 2 -> Habitación Penthouse"` **Explicación:** La llamada a `building.show()` desencadena una recursión que recorre todas las ramas y hojas. El resultado refleja la estructura de árbol completa definida en el Handler.

MEMENTO

Patrón de Diseño GoF

Problema en Airbnb

Permitir deshacer cambios en fechas de reserva sin violar el encapsulamiento.



Historial de Estados

```java
// MementoHandler.java
public class MementoHandler {
    public String demo() {
        Booking booking = new Booking("2026-06-01", "2026-06-05");
        BookingHistory history = new BookingHistory();

        history.save(booking.save()); // Guardar estado inicial
        booking.setCheckIn("2026-06-02"); // Modificar
        history.save(booking.save()); // Guardar nuevo estado

        booking.restore(history.undo()); // Deshacer ultimo cambio
        return booking.summary();
    }
}
```

- **Gestión de Flujo:** El Handler coordina al Originator (`Booking`) y al Caretaker (`BookingHistory`) para simular una operación de Undo.
- **Rol de Control:** Es el único que conoce cuándo se debe capturar un estado y cuándo restaurarlo.

```java
// Booking.java (Originator)
public class Booking {
    public BookingMemento save() { return new BookingMemento(checkIn, checkOut); }
    public void restore(BookingMemento m) { ... }
}
// BookingMemento.java (Memento)
public class BookingMemento {
    private final String checkIn; private final String checkOut;
}
// BookingHistory.java (Caretaker)
public class BookingHistory {
    private final Stack<BookingMemento> states = new Stack<>();
    public void save(BookingMemento m) { states.push(m); }
    public BookingMemento undo() { states.pop(); return states.peek(); }
}
```

Entrada (Request JSON)

```
{
  "pattern": "18",
  "input": "undo_test"
}
```

Resultado en Airbnb

`"2026-06-01 -> 2026-06-05"` **Explicación:**

El Handler guardó dos estados. Tras llamar a `undo()`, el Caretaker eliminó el estado actual y devolvió el anterior (el original). El Originator restauró sus campos privados a partir de ese memento.

PRESENTACIÓN FINAL

Reto de Clase: Patrón Memento

Descripción del Reto

Implementar un sistema de historial que permita navegar al menos 5 estados hacia atrás (Undo) y 5 hacia adelante (Redo) de forma consecutiva, manteniendo la integridad del objeto `Booking`.

- **Originator:** `Booking` (mantiene el estado de las fechas).
- **Memento:** `BookingMemento` (copia inmutable del estado).
- **Caretaker:** `BookingHistory` (gestiona dos pilas: `undoStack` y `redoStack`).

```java
public class BookingHistory {
    private final Stack<BookingMemento> undoStack = new Stack<>();
    private final Stack<BookingMemento> redoStack = new Stack<>();

    public void save(BookingMemento m) {
        undoStack.push(m);
        redoStack.clear(); // Nuevo cambio invalida el Redo
    }
    public BookingMemento undo(BookingMemento currentState) {
        if (undoStack.isEmpty()) return currentState;
        redoStack.push(currentState);
        return undoStack.pop();
    }
    public BookingMemento redo(BookingMemento currentState) {
        if (redoStack.isEmpty()) return currentState;
        undoStack.push(currentState);
        return redoStack.pop();
    }
}
```

```java
// MementoHandler.java - Fragmento de la Demo
for (int i = 0; i < 5; i++) {
    history.save(booking.save()); // Guardar estado actual
    booking.setCheckIn(newDates[i][0]);
    booking.setCheckOut(newDates[i][1]);
}

// Ejecución de 5 UNDOS
for (int i = 0; i < 5; i++) {
    booking.restore(history.undo(booking.save()));
}

// Ejecución de 5 REDOS
for (int i = 0; i < 5; i++) {
    booking.restore(history.redo(booking.save()));
}
```

Salida del Backend

```
--- FASE DE CAMBIOS ---
0. Inicial: 2026-06-01 -> 2026-06-05
1. Cambio: 2026-06-02 -> 2026-06-06 | 2. Cambio: 2026-06-03 -> 2026-06-07 | ...
5. Cambio: 2026-06-06 -> 2026-06-10

--- FASE DE UNDO (5 niveles) ---
Undo 1: 2026-06-05 -> 2026-06-09 | Undo 2: 2026-06-04 -> 2026-06-08
Undo 3: 2026-06-03 -> 2026-06-07 | Undo 4: 2026-06-02 -> 2026-06-06
Undo 5: 2026-06-01 -> 2026-06-05 (ESTADO INICIAL)

--- FASE DE REDO (5 niveles) ---
Redo 1: 2026-06-02 -> 2026-06-06 | Redo 2: 2026-06-03 -> 2026-06-07
Redo 3: 2026-06-04 -> 2026-06-08 | Redo 4: 2026-06-05 -> 2026-06-09
Redo 5: 2026-06-06 -> 2026-06-10 (ESTADO FINAL)
```

EVALUACIÓN

Catálogo de 23 Patrones GoF

Ítem de Evaluación	Excelente (5.0)	Bueno (4.0)	Aceptable (3.0)	Insuficiente (2.0)
Implementación	Código limpio, desacoplado.	Funciona pero hay acople.	Errores menores.	No funciona.
UML (PlantUML)	Refleja 100% el código.	Pequeñas discrepancias.	Diagrama genérico.	Sin diagrama.
Caso de Uso	Aplicado a Airbnb lógica.	Caso genérico.	Poco claro.	Sin sentido.
Explicación	Domina teoría y práctica.	Entiende la estructura.	Solo lee código.	No explica.

Nota Optativa

La nota final se asigna según la complejidad de la integración entre patrones y la robustez de la API Backend.

#	Patrón	Nota / 50
01	Singleton	50
02	Factory Method	41
03	Abstract Factory	40
04	Builder	44
05	Prototype	43
06	Adapter	43
07	Bridge	42
08	Composite	41
09	Decorator	50
10	Facade	48
11	Flyweight	41
12	Proxy	49

#	Patrón	Nota / 50
13	Chain of Responsibility	46
14	Command	40
15	Interpreter	40
16	Iterator	41
17	Mediator	43
18	Memento	43
19	Observer	48
20	State	49
21	Strategy	40
22	Template Method	48
23	Visitor	43

Cálculo del Promedio

$$\bar{x} = \frac{\sum_{i=1}^{23} n_i}{23} = \frac{1013}{23} = 44{,}04/50$$

Resultado Global

23 patrones implementados · Suma total: **1013 puntos**

Nota final: **44,04 / 50** $\Rightarrow$ **88,1%**

GRACIAS POR SU ATENCIÓN

Patrones GoF – Implementación Backend

Sofia Zambrano Paez / Kennet Mark Olarte

GitHub github.com/SophieZP/PatternDesign